Programming Assignment 2: Measuring Starlink LEO Satellite Network Infrastructure (20pts)

See Class Website/Gradescope For Due Date

1 Introduction

The goal of this assignment is to understand how traceroute measurements are used to analyze Low Earth Orbit (LEO) satellite network infrastructure. This assignment builds on the HitchHiking measurement methodology [2], which shows that probing Internet-exposed devices connected through LEO satellite networks can reveal properties of the underlying satellite infrastructure without requiring specialized hardware or direct access to satellite terminals. Unlike traditional terrestrial networks, LEO introduces additional latency, dynamic routing behavior, and geographically distributed points-of-presence (PoPs). These characteristics influence the paths packets take across the Internet.

In this assignment, you will analyze traceroute measurements collected from UCLA to destinations served by Starlink. Specifically, you will use the following two existing datasets:

- Traceroutes from UCLA to Starlink destinations
- Starlink prefix-to-PoP mapping

You will use the datasets to infer where traceroutes enter the Starlink network, identify latency jumps associated with the satellite segment, and analyze the relationship between Starlink PoPs and their corresponding destination IP addresses. More details are provided in the sections below.

1.1 Submission Instructions

There will be 4 tasks in this assignment, and you will complete each task in a separate `python` file. In total you will submit 4 files on Gradescope: `pa2q1.py`, `pa2q2.py`, `pa2q3.py`, `pa2q4.py`. Your scores will be immediately available upon submission and you have unlimited submission attempts until due date.

Separately, you will need to complete an AI-Assist report, where you describe if and how you used AI to help you complete this assignment. You can find the instructions on the class website. You will also need to answer two additional questions in this write up labeled **Written Response Question**. You will submit the report as a separate assignment on Gradescope.

2 Traceroute Measurements (4pts)

2.1 Background

Traceroute is a network measurement technique that discovers the sequence of routers traversed by packets sent between a source and destination. By analyzing the round-trip time (RTT) and IP addresses observed at each hop, we can infer properties of the underlying network path. You can review traceroute here [1].

In this assignment, you will analyze traceroutes collected toward Starlink-served destinations. You will identify where paths enter the Starlink network and detect latency jumps associated with the satellite segment.

2.2 Data Access

The traceroute dataset for this assignment is provided on the class website:

`https://lizizhikevich.github.io/ECE239AS-NetSec/HWs/input.json.gz`

You will download `input.json.gz` to complete the tasks in this section.

2.3 Data Format

The traceroute dataset contains newline-delimited JSON records. Each traceroute record contains metadata about the trace and a list of observed hops.

An example traceroute entry is shown below:


```
{
  "type":"trace",
  "src":"169.232.20.20",
  "dst":"116.91.210.100",
  "stop_reason":"COMPLETED",
  "hop_count":14,
  "hops":[
    {
      "addr":"169.232.20.1",
      "probe_ttl":1,
      "rtt":0.104
    },
    {
      "addr":"206.148.24.2",
      "probe_ttl":7,
      "rtt":133.599
    }
  ]
}
```


Explanation of relevant fields:

`src`: Source IP address

`dst`: Destination IP address

`stop_reason`: Whether the traceroute completed successfully
`hops`: List of responding hops
`addr`: IP address of a responding hop
`rtt`: Measured round-trip time in milliseconds

2.4 Data Processing

To analyze the traceroute dataset, you will need to do the following:

- Parse newline-delimited JSON traceroute records.
- Extract the list of responding hops from each traceroute.
- Compute RTT differences between consecutive responding hops.
- Identify Starlink hops using ASN information provided in the auxiliary mapping file. (See the following section for details)

2.5 Mapping IP addresses to AS numbers.

You will use the Routeviews Prefix-to-AS mappings for IPv4 to map an IP address to its prefix and AS number observed in BGP (referred to as the *CAIDA prefix2as dataset*). The file format is line-oriented, with one prefix-AS mapping per line.

The tab-separated fields are IP prefix, prefix length, and AS number. The dataset can be accessed on the CAIDA catalog at `https://www.caida.org/catalog/datasets/routeviews-prefix2as/`. You will use the `2026-05-01` snapshot of the dataset. The dataset will be store as the same directory as your python files, a.k.a, your file should read in `routeviews-rv2-20260501-1200.pfx2as.gz`. An example of prefix2as dataset is shown as below:

```
1.0.0.0 24 13335
1.0.7.0 24 38803
1.0.16.0 24 2519
1.0.64.0 18 18144
1.0.128.0 18 23969
```

You will use the longest prefix match (LPM) rule to match an IP address to its corresponding prefix. See Lecture 3 Slides for details on LPM. (`https://lizizhikevich.github.io/ECE239AS-NetSec/lectures/L3-Routing.pdf`) You can use an existing python libraries such as `pytricia` or `radix` to implement LPM.

Note that the autograder has limited memory. For the purposes of this assignment, you may selectively load the `prefix2as` file by reading only the lines associated with AS number `14593`.

Written Response Question 1: Can this selective loading approach cause incorrect prefix-to-AS matching? If so, explain why.

2.6 Task 1: Identify the Starlink network edge (2pts)

Your first task is to identify where each traceroute first enters the Starlink network. Your code is expected to do the following:

(1) Read the input traceroute file `input.json.gz`, which contains traceroutes from UCLA to Starlink destinations. The autograder will place the input file in the same directory it places your `python` file.

(2) Filter traceroutes based on `stop_reason`. Only keep traceroutes whose `stop_reason` is `COMPLETED`. Find the traceroute corresponding to the destination IP.

(3) Read the prefix2as file `routeviews-rv2-20260501-1200.pfx2as.gz`. Identify the first hop whose ASN belongs to Starlink (`ASN == '14593'`). The IP address of that hop is the Starlink ingress point IP.

(4) Print all unique Starlink ingress IPs into `output.txt`. The ordering of the IPs does not matter. The autograder expects the output file to be in the same directory as your `python` file. Example `output.txt`:

```
143.105.186.177
145.224.125.32
149.19.109.97
150.228.220.140
```

You will complete this task in `pa2q1.py`. Your code should be run by running the following command:

```
python3 pa2q1.py
```

2.7 Task 2: Identify the satellite link (2pts)

An Internet packet usually takes longer to traverse a satellite link than a terrestrial link. Your second task is to identify the satellite link by finding the largest RTT increase between consecutive responding hops in a traceroute. Your code is expected to do the following:

(1) Read the input traceroute file `input.json.gz` and compute RTT differences between consecutive responding hops.

(2) Identify the pair of hops `(IP1, IP2)` with the largest positive RTT increase.

(3) Make sure both `IP1` and `IP2` maps to the Starlink ASN (`'14593'`).

(5) Print unique pairs of `IP1, IP2` into `output.txt`. Example `output.txt`:

```
149.19.108.180,206.224.64.68
149.19.108.180,206.224.64.92
149.19.108.191,206.224.69.99
```

You will complete this task in `pa2q2.py`. Your code should be run by running the following command:

```
python3 pa2q2.py
```

3 Starlink PoPs (4pts)

3.1 Background

Starlink operates geographically distributed points-of-presence (PoPs) that connect satellite traffic into terrestrial infrastructure. Different destination IP prefixes may be served by different PoPs.

In this section, you will use a prefix-to-PoP mapping dataset to analyze which PoPs serve the largest number of destination IPs and which PoPs experience the largest RTT increases.

3.2 Data Access

The Starlink GeoIP dataset for this assignment is provided on the class website:

`https://lizizhikevich.github.io/ECE239AS-NetSec/HWs/starlink_geoip.csv.gz`

You will download `starlink_geoip.csv.gz` to complete the tasks in this section.

3.3 Data Format

The PoP mapping dataset contains mappings between IP prefixes and Starlink PoPs.

An example entry is shown below:

```
date,network,country_code,region_code,city,extra
2026-05-01,14.1.64.0/24,PH,PH-00,Manila,
2026-05-01,14.1.71.0/24,AU,AU-NT,Darwin,
2026-05-01,14.1.72.0/24,AU,AU-VIC,Melbourne,
2026-05-01,14.1.74.0/24,AU,AU-TAS,Hobart,
```

3.4 Data Processing

To analyze the PoP dataset, you will need to do the following:

- Parse IPv4 prefixes in CIDR notation.
- Perform longest-prefix matching to map destination IPs to PoP prefixes, country_code, and city
- Aggregate traceroute statistics by PoP, defined as a pair of (country_code, city).

3.5 Task 3: Find the most popular PoPs (2pts)

Some PoPs serve more customers than others, depending on region, population density, and other reasons. Your third task is to identify the PoPs serving the largest number of destination IP addresses. Your code is expected to do the following:

(1) Read the input traceroute file `input.json.gz`. Map each traceroute destination IP to a Starlink PoP using longest-prefix match.

(2) Find unique destination IPs served by each PoP prefix.

(3) Count the number of unique destination IPs per PoP. We define each PoP as its (country_code, city).

(4) Print the top 20 PoPs and their destination IP counts into `output.txt`, one per line, separated by a comma. Example `output.txt`:

```
MX,Mexico City,3173
CO,Bogota,2595
```

You will complete this task in `pa2q3.py`. Your code should be run by running the following command:

```
python3 pa2q3.py
```

3.6 Task 4: Find the PoPs most distant from their customers (2pts)

Some PoPs are more remote to their customers compared to others. Customers whose PoPs are far away may experience higher latency compared to customers who are closer to their PoPs. Your final task is to identify the Starlink PoPs that have higher delta RTT in their corresponding satellite hop. Your code is expected to do the following:

(1) Read the input traceroute file `input.json.gz`. For each traceroute, identify the satellite link delta RTT (from task 2), and identify the destination PoP.

(2) Group traceroutes by destination PoP.

(3) Compute the median satellite link delta RTT for each PoP.

(4) Print top 20 PoP with the largest median RTT jump and its median RTT jump into `output.txt`. Round the median RTT values to 2 decimals when printing. Example `output.txt`:

```
CA,Kuujjuaq,527.84
MH,Majuro,122.45
PN,Adamstown,95.34
FK,Stanley,91.11
MN,Ulaanbaatar,67.35
```

You will complete this task in `pa2q4.py`. Your code should be run by running the following command:

```
python3 pa2q4.py
```

Written Response Question 2: Visualize the distribution of satellite link delta RTTs for different Starlink PoPs. You may use any visualization style you prefer. Based on your visualization, discuss which PoPs appear to be more distant from their customers.

References

[1] A-Line Cloud. How traceroute works: A visual, interactive guide. `https://cloud.a-line.be/blog/how-traceroute-works`. Accessed: 2026-05-18.

[2] Liz Izhikevich, Manda Tran, Katherine Izhikevich, Gautam Akiwate, and Zakir Durumeric. Democratizing leo satellite network measurement. In *Abstracts of the 2024 ACM SIGMETRICS/IFIP PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems*, pages 15–16. Association for Computing Machinery, 2024.